

SENG 204: DATA STRUCTURES

Assoc. Prof. Dr. Hilal ARSLAN

Week 1: General Information

Instructor: Assoc. Prof. Dr. Hilal Arslan

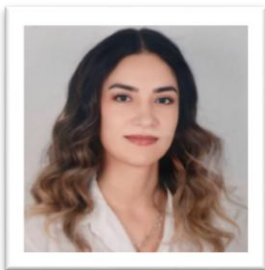
Email: hilalarslanceng@gmail.com



Course Schedule: Wed, 14:30, A303

The normal flow of this course has 2 hours (theoretical) lectures
+ 2 hours of laboratory work

Teaching Assistant: Büşra Yıldırım, Pelin Nur Çöl



About the Course

- Introduces fundamental concepts of **data organization** and **storage**
- Explains how data structures enable **efficient data access and processing**
- Covers core **linear structures**: arrays, linked lists, stacks, and queues
- Presents **non-linear structures**: trees, graphs, and hash tables
- Teaches algorithm efficiency through time and space complexity analysis
- Focuses on selecting appropriate data structures for real-world problems
- Develops skills for designing scalable and high-performance software systems

Data Structures

“I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.”

Linus Torvalds, 2006

Text Book and References

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (3rd ed.), Addison Wesley, 2006 (*Current Textbook*).
2. M. T. Goodrich, R. Tamassia and D. Mount, *Data Structures and Algorithms in C++*, John Wiley & Sons, 2004 Mark Allen Weiss, *Data Structures and Problem Solving Using C++*, 2nd ed., Addison Wesley 2000
3. Sartaj Sahni, *Data Structures, Algorithms, and Applications in C++*, McGraw Hill, 1998.
4. H.M. Deitel, P.J. Deitel, *How To Program C++ and Java by* Prentice-Hall, 2001.

Outline

- Introduction to C++
- Algorithm analysis
- Sorting
- Listed Lists
- Stacks
- Queues
- Introduction to Trees and Binary Trees
- Binary Search Trees and AVL Trees
- Heaps
- Hashing
- Graphs

Grading

- Midterm: 30%
- Lecture Quizzes: 20%
- Lab Exam: 20%
- Final: 30%

Overview

Data Structures

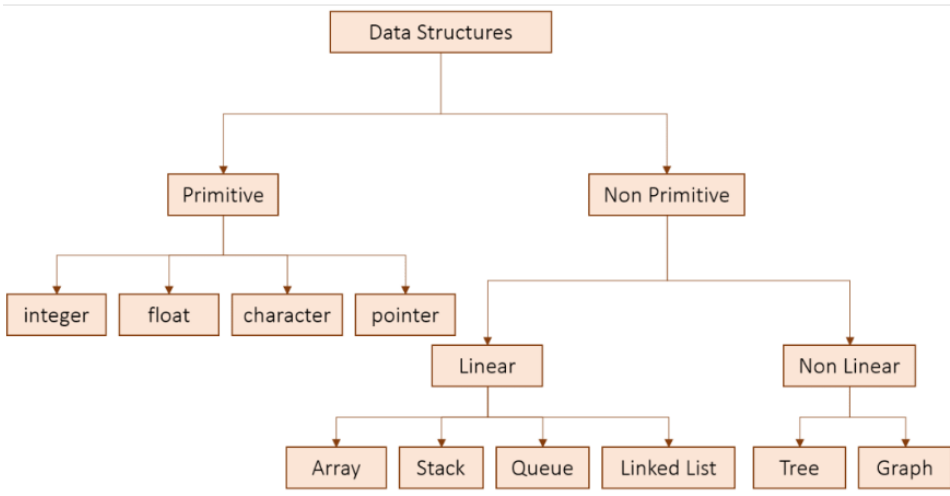
Definition:

- A data structure is a particular way of storing and organizing data in a computer. In other words the organised collection of data is known as data structure.
- Data structure = organised data + operations
- Common data structures are arrays, linked lists, queues, stacks, binary trees, and hash tables.

Classification of data structures:

1. Primitive data structure
2. Non primitive data structure

Data Structures







Data

Digital Data



Movies



Music



Photos



Protein
Shapes



Maps

DNA

```
gatctttt  tttaacgt  cttctttt  gatctttt  taggtcatg  atccctgtg  
gataagtgt  tattcacatg  gcagatccta  taattatga  ggtatgttg  ttgtgaatga  
ccgtgatcg  tattgagat  aagctgggat  ctaaatgga  ttttatgaa  agtaaatcga  
cagaatcag  gttgttatgt  ggaatatctac  tggttttac  ctgctttta  gcatagttat  
sosaatttgt  togogagat  tttagatga  tttagatga  ttaataaat  otttgaaac
```

0010101001010101010010010010101000010010010100....

How do we organize information so that we can find, update, add, and delete portions of it efficiently?

Digital Data Must Be ...

- **Encoded** (e.g. 01001001 $\leftrightarrow$ )

- **Arranged**

- Stored in an orderly way in memory / disk

- **Accessed**

- Insert new data
 - Remove old data
 - Find data matching some condition

} The focus of
this class

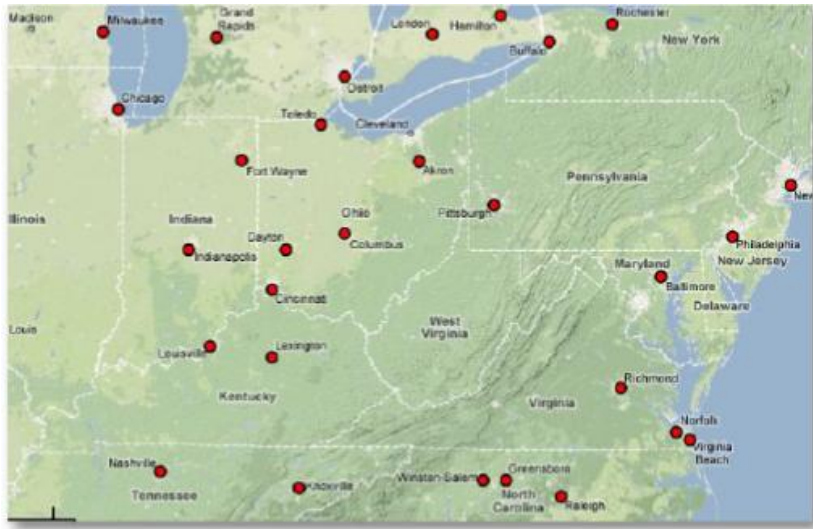
- **Processed**

- Algorithms: shortest path, minimum cut, FFT, ...

Data Structure Example Applications

- How does Google quickly find web pages that contain a search term?
- How can a subsequence of DNA be quickly found within the genome?
- How does your operating system track which memory (disk or RAM) is free?

Data Structure Example Applications



Basic Terminology

- A **program** should give **correct results**, but it should also run **efficiently**
- A program is considered **efficient** when it executes in **minimum time** and uses **minimum memory space**.
- In order to write efficient programs we need to apply certain data management concepts.
- **Data structure** is a crucial part of data management.
- A **data structure** is basically a group of data elements that are put together under one name, and which defines a particular way of storing and organizing data in a computer so that it can be used efficiently

Basic Terminology

- Data structures are used in almost every program or software system.
- Some common examples of data structures are arrays, linked lists, queues, stacks, binary trees, and hash tables.
- The primary goal of a program is not to perform calculations or operations but to store and retrieve information as fast as possible.

Basic Terminology

- There are different types of data structures that the C/C++ language supports.
- While one type of data structure may permit adding of new data items only at the beginning, the other may allow it to be added at any position.
- While one data structure may allow accessing data items sequentially, the other may allow random access of data.
- So, selection of an appropriate data structure for the problem is a crucial decision and may have a major impact on the performance of the program.